

*****Pornind de la aplicația exemplu din laboratorul 10, să se modifice aceasta astfel încât să permită primirea de evaluări între 1-5 privind calitatea serviciului și să salveze numele utilizatorului și evaluarea într-un fișier local. Se vor utiliza microservicii Python și o comunicare prin RabbitMQ.

Dacă se consideră necesar se mai pot crea oricâte servicii se dorește conform gândirii studentului. Se vor respecta principiile SOLID pentru microservicii. La prezentare se va arăta diagrama de clase și se va explica aceasta împreună cu gradul de respectare a principiilor. Apoi va urma discuția pe cod.

LAB 10 OKAZII V2 in python la care adaugam urm chestii, KAFKA INSTALL ai instructiuni PREA MULT IMPROVIZEZ CRECA

AUCTIONEER.PY

```
from kafka import KafkaConsumer, KafkaProducer
from Decorators import rating_auction_person, RatingAuctionPerson
```

@rating_auction_person # Varianta 2

class Auctioneer:

```
    def __init__(self, bids_topic, notify_message_processor_topic):
        super().__init__()
        self.bids_topic = bids_topic
        self.notify_processor_topic = notify_message_processor_topic
```

consumatorul pentru ofertele de la licitatie

```
self.bids_consumer = KafkaConsumer(
    self.bids_topic,
    auto_offset_reset="earliest", # mesajele se preiau de la cel mai vechi la cel mai recent
    group_id="auctioneers",
    consumer_timeout_ms=15_000 # timeout de 15 secunde
)
```

producatorul pentru notificarea procesorului de mesaje

```
self.notify_processor_producer = KafkaProducer()
```

def receive_bids(self):

```
    # se preiau toate ofertele din topicul bids_topic
    print("Astept oferte pentru licitatie...")
    for msg in self.bids_consumer:
        for header in msg.headers:
            if header[0] == "identity":
                identity = str(header[1], encoding="utf-8")
```

```

        elif header[0] == "amount":
            bid_amount = int.from_bytes(header[1], 'big')

    print("{} a licitat {}".format(identity, bid_amount))

    # bids_consumer genereaza exceptia StopIteration atunci cand se atinge timeout-ul de 10
    secunde
    # => licitatie se incheie dupa ce timp de 15 secunde nu s-a primit nicio oferta
    self.finish_auction()

    def finish_auction(self):
        print("Licitatie s-a incheiat!")
        self.bids_consumer.close()

        # se notifica MessageProcessor ca poate incepe procesarea mesajelor
        auction_finished_message = bytearray("incheiat", encoding="utf-8")
        self.notify_processor_producer.send(topic=self.notify_processor_topic,
        value=auction_finished_message)
        self.notify_processor_producer.flush()
        self.notify_processor_producer.close()

    def run(self):
        self.receive_bids()

if __name__ == '__main__':
    auctioneer = Auctioneer(
        bids_topic="topic_oferte",
        notify_message_processor_topic="topic_notificare_procesor_mesaje"
    )
    #auctioneer = RatingAuctionPerson(auctioneer) # Varianta 1
    auctioneer.run()

```

 BIDDER.PY

```

from kafka import KafkaProducer, KafkaConsumer
from random import randint
from uuid import uuid4
from Decorators import rating_auction_person, RatingAuctionPerson

```

```

@rating_auction_person # Varianta 2
class Bidder:
    def __init__(self, bids_topic, result_topic):

```

```

super().__init__()
self.bids_topic = bids_topic
self.result_topic = result_topic

# producatorul pentru oferte de licitatie
self.bid_producer = KafkaProducer()

# consumatorul pentru rezultatul licitatiei
self.result_consumer = KafkaConsumer(
    self.result_topic,
    auto_offset_reset="earliest" # mesajele se preiau de la cel mai vechi la cel mai recent
)

self.my_bid = randint(1000, 10_000) # se genereaza oferta ca numar aleator intre 1000 si
10.000
self.my_id = uuid4() # se genereaza un identificator unic pentru ofertant

def bid(self):
    # se construiesc mesajul pentru licitare
    print("Trimit licitatie mea: {}".format(self.my_bid))
    bid_message = bytearray("licitez", encoding="utf-8") # corpul contine doar mesajul "licitez"
    bid_headers = [ # antetul mesajului contine identitatea ofertantului si, respectiv, oferta sa
        ("amount", self.my_bid.to_bytes(2, byteorder='big')),
        ("identity", bytes("Bidder {}".format(self.my_id), encoding="utf-8"))
    ]

    # se trimite licitatie sub forma de mesaj catre Kafka
    self.bid_producer.send(topic=self.bids_topic, value=bid_message, headers=bid_headers)

    # exista o sansa din 2 ca oferta sa fie trimisa de 2 ori pentru a simula duplicatele
    if randint(0, 1) == 1:
        self.bid_producer.send(topic=self.bids_topic, value=bid_message,
headers=bid_headers)

    self.bid_producer.flush()
    self.bid_producer.close()

def get_winner(self):
    # se asteapta raspunsul licitatiei
    print("Astept rezultatul licitatiei...")
    result = next(self.result_consumer)

    # se verifica identitatea castigatorului
    for header in result.headers:

```

```

        if header[0] == "identity":
            identity = str(header[1], encoding="utf-8")

        if identity == "Bidder {}".format(self.my_id):
            print("{} Am castigat!!!".format(self.my_id))
        else:
            print("{} Am pierdut...".format(self.my_id))

        self.result_consumer.close()

    def run(self):
        self.bid()
        self.get_winner()

if __name__ == '__main__':
    bidder = Bidder(
        bids_topic="topic_oferte",
        result_topic="topic_rezultat"
    )
    # bidder = RatingAuctionPerson(bidder) # Varianta 1
    bidder.run()

```

BIDDINGPROCESSOR.PY

```

from kafka import KafkaConsumer, KafkaProducer
from Decorators import rating_auction_person, RatingAuctionPerson

@rating_auction_person # Varianta 2
class BiddingProcessor:
    def __init__(self, processed_bids_topic, result_topic):
        super().__init__()
        self.processed_bids_topic = processed_bids_topic
        self.result_topic = result_topic

        # consumatorul pentru ofertele procesate
        self.processed_bids_consumer = KafkaConsumer(
            self.processed_bids_topic,
            auto_offset_reset="earliest", # mesajele se preiau de la cel mai vechi la cel mai recent
            consumer_timeout_ms=3000
        )

        # producatorul pentru trimiterea rezultatului licitatiei
        self.result_producer = KafkaProducer()

```

```

def get_processed_bids(self):
    # se preiau toate ofertele procesate din topicul processed_bids_topic
    print("Astept ofertele procesate de MessageProcessor...")

    # ofertele se stocheaza sub forma de perechi <PRET_LICITAT, MESAJ_OFERTA>
    bids = dict()
    no_bids_available = True

    while no_bids_available:
        for msg in self.processed_bids_consumer:
            for header in msg.headers:
                if header[0] == "amount":
                    bid_amount = int.from_bytes(header[1], 'big')
                    bids[bid_amount] = msg

    # daca inca nu exista oferte, se asteapta in continuare
    if len(bids) != 0:
        no_bids_available = False

    self.processed_bids_consumer.close()
    self.decide_auction_winner(bids)

def decide_auction_winner(self, bids):
    print("Procesez ofertele...")

    if len(bids) == 0:
        print("Nu exista nicio oferta de procesat.")
        return

    # sortare dupa oferte, descrescator
    sorted_bids = sorted(bids.keys(), reverse=True)

    # castigatorul este ofertantul care a oferit pretul cel mai mare
    winner = bids[sorted_bids[0]]

    for header in winner.headers:
        if header[0] == "identity":
            winner_identity = str(header[1], encoding="utf-8")

    print("Castigatorul este:")
    print("{} - pret licitat: {}".format(winner_identity, sorted_bids[0]))

```

```

        # se trimite rezultatul licitatiei pentru ca entitatile Bidder sa il preia din topicul
        corespunzator
        self.result_producer.send(topic=self.result_topic, value=winner.value,
headers=winner.headers)
        self.result_producer.flush()
        self.result_producer.close()

    def run(self):
        self.get_processed_bids()

if __name__ == '__main__':
    bidding_processor = BiddingProcessor(
        processed_bids_topic="topic_oferte_procesate",
        result_topic="topic_rezultat"
    )
    # bidding_processor = RatingAuctionPerson(bidding_processor) # Varianta 1
    bidding_processor.run()

```

DECORATORS.PY

```

from copy import copy
import random
from mq_communication import RabbitMq

```

```

def rate():
    rabbit_mq = RabbitMq()
    rating = f'{random.randint(1, 5)}'
    print(f'Rating: {rating}')
    rabbit_mq.send_message(rating)

```

Varianta 1: Design Pattern-ul decorator

```

class RatingAuctionPerson:
    def __init__(self, auction_person):
        self.__delegate = auction_person

    def run(self):
        self.__delegate.run()
        rate()

```

Varianta 2: Decoratori speciali Python

```
def rating_auction_person(person):  
    rabbit_mq = RabbitMq()  
    old_run = copy(person.run) # Daca nu fac o copie a functiei, atunci functia run va crea o  
    bucla infinita...
```

```
    def decorated_run(self):  
        old_run(self)  
        rate()
```

```
    person.run = decorated_run
```

```
    return person
```

MESSAGEPROCESSOR.PY

```
from datetime import datetime  
from kafka import KafkaConsumer, KafkaProducer  
from Decorators import rating_auction_person, RatingAuctionPerson
```

@rating_auction_person # Varianta 2

class MessageProcessor:

```
    def __init__(self, bids_topic, notify_message_processor_topic, processed_bids_topic):  
        super().__init__()  
        self.bids_topic = bids_topic  
        self.notify_message_processor_topic = notify_message_processor_topic  
        self.processed_bids_topic = processed_bids_topic
```

```
    # consumatorul notificarii de la Auctioneer cum ca s-a terminat licitatie  
    self.notify_message_processor_consumer = KafkaConsumer(  
        self.notify_message_processor_topic,  
        auto_offset_reset="earliest" # mesajele se preiau de la cel mai vechi la cel mai recent  
    )
```

```
    # consumatorul pentru ofertele de la licitatie  
    self.bids_consumer = KafkaConsumer(  
        self.bids_topic,  
        auto_offset_reset="earliest",  
        consumer_timeout_ms=1000  
    )
```

```
    # producatorul pentru mesajele procesate  
    self.processed_bids_producer = KafkaProducer()
```

```

    # ofertele se pun in dictionar, sub forma de perechi <IDENTITATE_OFERTANT,
    MESAJ_OFERTA>
    self.bids = dict()

def get_and_process_messages(self):
    # se asteapta notificarea de la Auctioneer pentru incheierea licitatiei
    print("Astept notificare de la toate entitatile Auctioneer pentru incheierea licitatiei...")
    auction_end_message = next(self.notify_message_processor_consumer)

    # a ajuns prima notificare, se asteapta si celelalte notificari timp de maxim 15 secunde
    self.notify_message_processor_consumer.config["consumer_timeout_ms"] = 15_000
    for auction_end_message in self.notify_message_processor_consumer:
        pass
    self.notify_message_processor_consumer.close()

    if str(auction_end_message.value, encoding="utf-8") == "incheiat":
        # se preiau toate ofertele din topicul bids_topic si se proceseaza
        print("Licitatie incheiata. Procesez mesajele cu oferte...")

        for msg in self.bids_consumer:
            for header in msg.headers:
                if header[0] == "identity":
                    identity = str(header[1], encoding="utf-8")

                    # eliminare duplicate
                    if identity not in self.bids.keys():
                        self.bids[identity] = msg

        self.bids_consumer.close()

        # sortare dupa timestamp
        sorted_bids = sorted(self.bids.values(), key=lambda bid: bid.timestamp)

        self.finish_processing(sorted_bids)

def finish_processing(self, sorted_bids):
    print("Procesarea s-a incheiat! Trimit urmatoarele oferte:")
    for bid in sorted_bids:
        for header in bid.headers:
            if header[0] == "identity":
                identity = str(header[1], encoding="utf-8")
            elif header[0] == "amount":
                bid_amount = int.from_bytes(header[1], 'big')

```



```

        print("{} {} a licitat {}".format(datetime.fromtimestamp(bid.timestamp / 1000), identity,
        bid_amount))

        # se stocheaza mesajele ordonate dupa timestamp si fara duplicate intr-un topic separat
        self.processed_bids_producer.send(topic=self.processed_bids_topic, value=bid.value,
        headers=bid.headers)

        self.processed_bids_producer.flush()
        self.processed_bids_producer.close()

    def run(self):
        self.get_and_process_messages()

if __name__ == '__main__':
    message_processor = MessageProcessor(
        bids_topic="topic_oferte",
        notify_message_processor_topic="topic_notificare_procesor_mesaje",
        processed_bids_topic="topic_oferte_procesate"
    )
    # message_processor = RatingAuctionPerson(message_processor) # Varianta 1
    message_processor.run()

```

mq_communication.py

```

import pika
from retry import retry

```

```

class RabbitMq:
    config = {
        'host': '0.0.0.0',
        'port': 5678,
        'username': 'student',
        'password': 'student',
        'exchange': 'examenp6.direct',
        'routing_key': 'examenp6.routingKeyToRatingCollector',
        'queue': 'examenp6.queueToRatingCollector'
    }
    credentials = pika.PlainCredentials(config['username'], config['password'])
    parameters = (pika.ConnectionParameters(host=config['host']),
        pika.ConnectionParameters(port=config['port']),
        pika.ConnectionParameters(credentials=credentials))

```

```

def __init__(self):
    self.__on_receive_message_callback = None

def on_received_message(self, blocking_channel, deliver, properties,
                        message):
    result = message.decode('utf-8')
    blocking_channel.confirm_delivery()
    try:
        if self.__on_receive_message_callback is not None:
            self.__on_receive_message_callback(result)
    except Exception as e:
        print(e)
    finally:
        blocking_channel.stop_consuming()

@retry(pika.exceptions.AMQPConnectionError, delay=5, jitter=(1, 3))
def receive_message(self):
    # automatically close the connection
    with pika.BlockingConnection(self.parameters) as connection:
        # automatically close the channel
        with connection.channel() as channel:
            channel.basic_consume(self.config['queue'],
                                  self.on_received_message,
                                  auto_ack=True)

            try:
                channel.start_consuming()
            # Don't recover connections closed by server
            except pika.exceptions.ConnectionClosedByBroker:
                print("Connection closed by broker.")
            # Don't recover on channel errors
            except pika.exceptions.AMQPChannelError:
                print("AMQP Channel Error")
            # Don't recover from KeyboardInterrupt
            except KeyboardInterrupt:
                print("Application closed.")
                exit(0)

def send_message(self, message):
    # automatically close the connection
    with pika.BlockingConnection(self.parameters) as connection:
        # automatically close the channel
        with connection.channel() as channel:
            self.clear_queue(channel)
            channel.basic_publish(exchange=self.config['exchange'],

```

```
        routing_key=self.config['routing_key'],
        body=message)
```

```
def clear_queue(self, channel):
    channel.queue_purge(self.config['queue'])
```

```
def set_on_receive_message_callback(self, callback):
    self.__on_receive_message_callback = callback
```

PrepareAuction.py

```
from kafka import KafkaAdminClient
from kafka import KafkaConsumer
from kafka.admin import NewTopic
import time
```

```
if __name__ == '__main__':
    admin = KafkaAdminClient()
```

```
    used_topics = (
        "topic_oferte",
        "topic_rezultat",
        "topic_oferte_procesate",
        "topic_notificare_procesor_mesaje",
    )
```

```
# se sterg topic-urile, daca exista deja
print("Se sterg topic-urile existente...")
```

```
kafka_topics = KafkaConsumer().topics()
for topic in kafka_topics:
    if topic in used_topics:
        print("\tSe sterge {}".format(topic))
        admin.delete_topics(topics=[topic], timeout_ms=2000)
```

```
# se asteapta putin ca stergerea sa aiba loc
time.sleep(2)
```

```
# se creeaza topic-urile necesare aplicatiei
print("Se creeaza topic-urile necesare:")
```

```
lista_topici = [
    NewTopic(name=used_topics[0], num_partitions=4, replication_factor=1),
    NewTopic(name=used_topics[1], num_partitions=1, replication_factor=1),
    NewTopic(name=used_topics[2], num_partitions=1, replication_factor=1),
```

```

        NewTopic(name=used_topics[3], num_partitions=1, replication_factor=1)
    ]
    for topic in lista_topicuri:
        print("\t{}".format(topic.name))
    admin.create_topics(lista_topicuri, timeout_ms=3000)

    print("Gata! Microserviciile participante la licitatie pot fi pornite.")
    -----
RatingsProcessor.py

from mq_communication import RabbitMq

class RatingsProcessor:
    def __init__(self):
        self.__rabbit_mq = RabbitMq()
        self.__ratings = []
        self.__rabbit_mq.set_on_receive_message_callback(self.__receive_message)

    def __compute_mean(self):
        sum = 0
        for rating in self.__ratings:
            sum += rating
        return sum / len(self.__ratings)

    def __receive_message(self, message):
        print(f'Rating primit: {message}')
        self.__ratings.append(int(message))
        open('logs.txt', 'w').write(f'{self.__compute_mean()}')

    def run(self):
        while True:
            self.__rabbit_mq.receive_message()

if __name__ == '__main__':
    RatingsProcessor().run()

```